

CS339: Abstractions and Paradigms for Programming

Introduction and Logistics

Manas Thakur
CSE, IIT Bombay



Autumn 2026

Natural Languages

- Name the languages that you can speak/read/write.
 - *The APP course in no other country can start with this slide!*
- Which language do you think in?
- Have you tried learning a new language?
- How different was that language from those you already knew?
- What was your strategy?



Programming Languages

- Name the PLs that you know.
 - What do we mean by *know*?
 - Name an English letter that's not a PL.
 - Why do we have so many PLs?
 - How did so many PLs evolve?
- Or aren't they so many, but just syntactic sugars over a set of fundamental *computational constructs*?

If languages were cabs

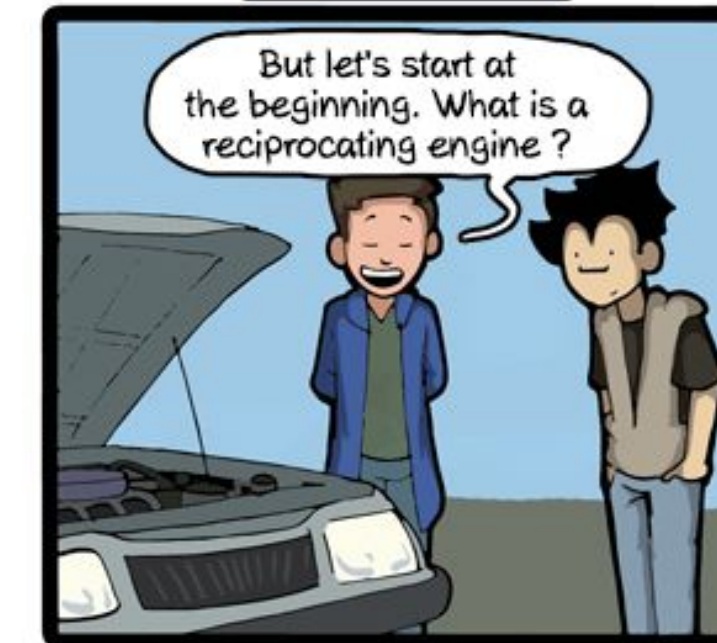
PHP



Javascript



Assembly



JAVA



Objective-C



Open Source

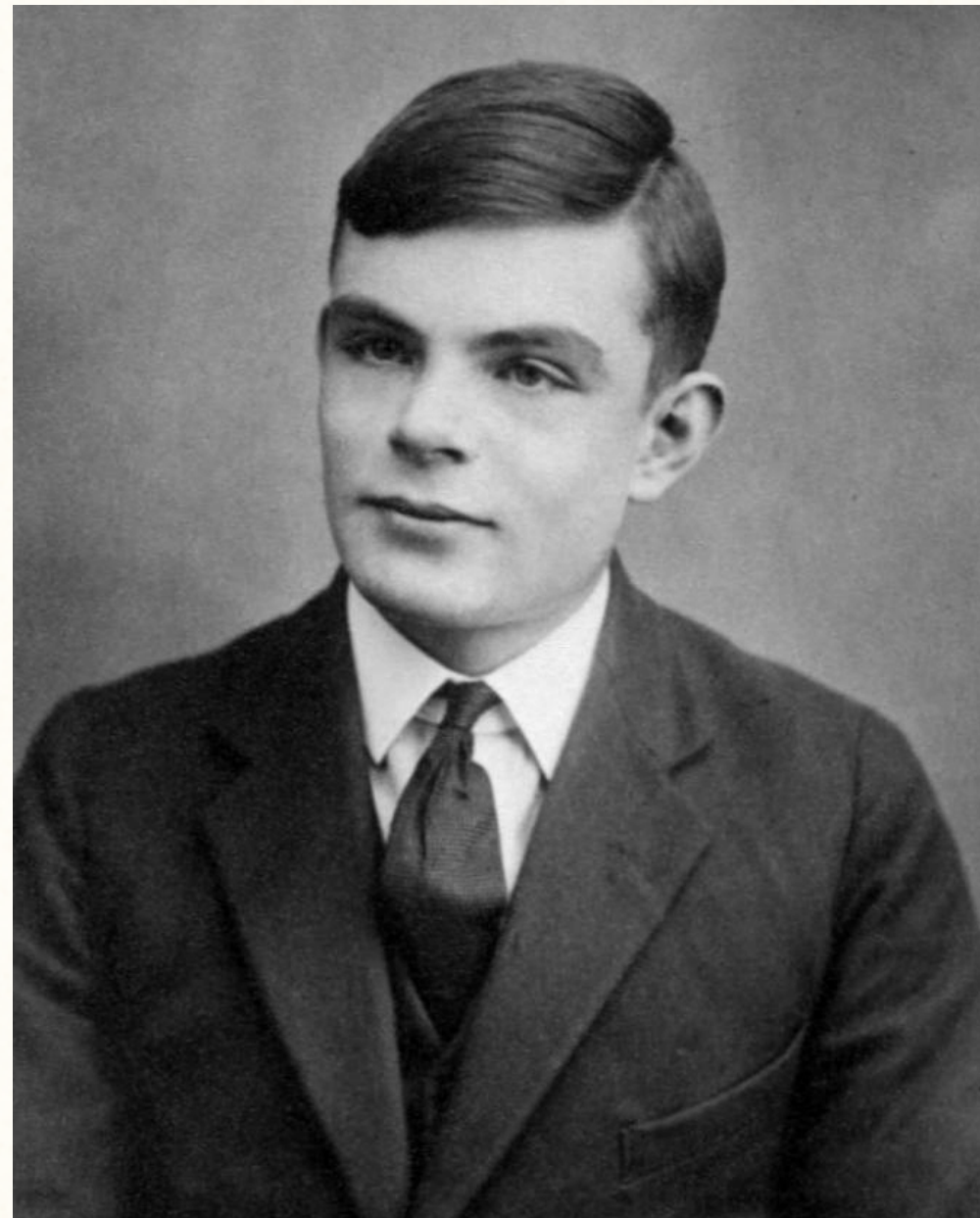


CommitStrip.com



Standing on the shoulders of giants

➤ Who are these people?



Alan Turing



Alonzo Church

➤ Hints:

- Both were computer scientists.
- The most famous award in CS is named after the LHS.
- RHS was the PhD advisor of the LHS.

Two models of computation

- The **Turing Machine** (1936) performs computations by:

- Reading input
- Modifying an internal memory using instructions
- Producing output

If you don't remember TOC,
think of a computer that you are used to!!

- The **Lambda Calculus** (1936) performs computations by:

- Evaluating expressions

We would learn more of it in this course.

- **Church-Turing Thesis** (1937): Both the models, and all other reasonable computation models, are equivalent!



Abstractions and Paradigms for Programming

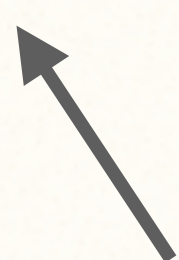
- **WHAT** is the square root of a number X ?

A number Y such that $Y * Y == X$.

- **HOW** to compute the square root of a number X ?

- Make a guess Y .
- Improve the guess Y until its square is near enough to X .

A SQRT “procedure”



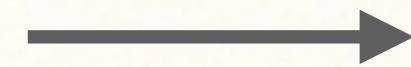
1. Procedural Abstraction

- Can we reuse the SQRT procedure as a blackbox?

Compute $\text{SQRT}(A) + \text{SQRT}(B)$.

- Can we consume and produce procedures?

Procedure
 $Y \longrightarrow \text{AVG}(Y, X/Y)$



Procedure
FIXED-POINT (F)



Procedure
SQRT(X)

Functional
Paradigm

- Functions become real entities in our programs!

2. Data Abstraction

➤ Say we have a general procedure to add two things and multiply by another:

Procedure
 $(X + Y) * Z$

➤ Can X, Y, Z be

- numbers?
- complex numbers?
- vectors?
- matrices?
- polynomials?
- signals?

Object-Oriented
Paradigm

Types &
Proofs

Stream
Processing

3. Language Abstraction

- Can we **design a language** that gives us higher level primitives?
 - vectors? matrices? tensors?
 - images? videos?
- Can we **modify an existing language** to work differently?
 - lazy evaluation instead of eager?
 - automatic allocation instead of explicit types?
- Can we **extend a language** to support something more?
 - C to C++?

Logic
Paradigm

Interpreter
Design



Abstractions and Paradigms for Programming

- Computation is much more fundamental (and older) than computers or computer science, and is very much about specifying the right abstractions.
 - There are only a handful of paradigms (aka *ways*) of specifying computations.
 - Much fewer than there are programming languages.
-
- There is no way we can teach you the PL that you can keep using for the next 30-40 years.
 - But we would teach you the ways of specifying and abstracting computations, and equip you with the skills to design your own PLs, and set you on track to become a **successful computing engineer**.



A Pertinent Question

Today, almost every introductory programming assignment can be solved by an LLM.

So why are we here?

aka **What is the role of computing engineers in the modern world?**

Something an APPer would be able to understand the best! 😎



The APP Course

- Empowers us to **choose** the right PL for the task at hand;
- Makes **learning** a new PL much easier;
- Equips us in **designing** a new PL and its interpreter;
- Is huge **fun**!

Side learnings:

- Ability to **comprehend large programs**
- **Language technologies** that everyone uses
- Crux of **future** programming languages
- Ability to **audit/verify AI-generated code**, not just produce it
- Multiple **equally powerful** ways of thinking about the same thing
- Motivation to be able to **cook** (CS302) and **process** (CS614, CS616, CS618, CS6004) the **food** that everyone (CSXYZ) eats



Logistics, Evaluation, Rules

CS339 · CS355

Abstractions and Paradigms for Programming · Programming Paradigms Lab

Autumn 2026

Manas Thakur, CSE, IIT Bombay

LOGISTICS

Lectures (CS339)	Mon, Tue, Thu · Slot 3 · 11:35–12:30
Lab (CS355)	Thu · Slot L3 · 14:00–17:00 (staggered — see below)
Office Hour	Wed · 16:00–17:00 · CC 308, Computing Complex
Instructor contact	manas@cse.iitb.ac.in

Lab staggering: Each week the class is split into two batches. Batch A works 14:00–15:25, and Batch B works 15:30–16:55. The two batches receive separate lab questions, so please arrive within your assigned slot.

EVALUATION — CS339

Component	Marks
Quiz 1	10
Quiz 2	10
Midsem	30
Endsem	40
Participation	5
Attendance	5

Attendance marks (based on % of CS339 lectures attended):

Attendance	Marks
> 90%	5
> 80%	4
> 70%	3
> 60%	2
> 50%	1

If you are among the top 25% scorers (by cumulative marks at the time of computation), each threshold above is relaxed by 20 percentage points — e.g. 90%→70%, 80%→60%, 70%→50%, 60%→40%, 50%→30%.

If your attendance is > 80%, you get up to 2 extra participation marks — but total participation is still capped at 5.

EVALUATION — CS355

Component	Marks
Labs — best 7 of 8 (4 marks each)	28
Lab Midsem	35
Lab Endsem	35
Total	98

Out of 8 labs, you may skip 1 without penalty (only your best 7 count toward the 28). Attending all 8 earns 2 bonus marks, taking the maximum possible to 100.

Course Handout

Quick links & policies · Autumn 2026 · CS339 / CS355

QUICK LINKS

Course Plan & Schedule	Instructor Webpage
 Running plan, tentative dates, topic-wise breakup (Google Sheet)	 www.cse.iitb.ac.in/~manas

The evaluation scheme above is final, but topic order/dates on the schedule may shift slightly as the semester progresses. Refer to Moodle for all other links and details.

ACADEMIC INTEGRITY

- All submitted work — quizzes, exams, and lab assignments — must be entirely your own.
- Unauthorized collaboration, copying (from peers, the internet, or previous years' solutions), and use of unpermitted aids during evaluations are treated as violations of academic integrity.
- A first violation typically results in a zero on the component involved; repeated or severe violations will be referred to the Institute's Anti-Copying/Unfair-Means process and may lead to an FR grade, per Institute policy.
- If in doubt about whether something is permitted (e.g., a collaboration or a resource), ask the instructor or TAs before submitting — not after.

Use of AI tools (ChatGPT, Claude, Copilot, and similar) is **not permitted** during CS355 labs or in any CS339 exam (quizzes, midsem, endsem). Using such tools during these evaluations will be treated as an academic-integrity violation.

GOOD TO REMEMBER

- Attendance and marks will be maintained on Moodle — check regularly and flag discrepancies within one week.
- Lab marks are posted on Moodle as a combined total for each half (Labs 1–4 and Labs 5–8) before the lab midsem and lab endsem respectively.
- Regrading requests must be submitted via a Google Form (separate form will be provided for each evaluation) before the stated deadline.
- Missed Quiz 1 for a valid medical (pink slip) or institute approved reason? Its marks are scaled from the Midsem. Missed Quiz 2? Its marks are scaled from the Endsem. Supporting documents must be uploaded to a Google form linked on Moodle before the exam's date.
- Missed a lab? No documentation is needed for missing a single lab — the best-7-of-8 scheme already accounts for it. For missing other labs and classes, valid documented reasons as per the previous point will be applicable.
- Keep this sheet handy through the semester — it will be up on Moodle but the print will not be re-circulated.

COMMUNICATION

- Discussions are encouraged — during labs, office hours, and on the Moodle forum.
- Collaborations are discouraged – with students as well as with AI – unless explicitly asked or allowed.
- Do not DM the TAs;** only the information conveyed by the instructor will be considered valid.

Questions? Reach out in office hours (Wed 16:00–17:00, CC 308) or via email — manas@cse.iitb.ac.in



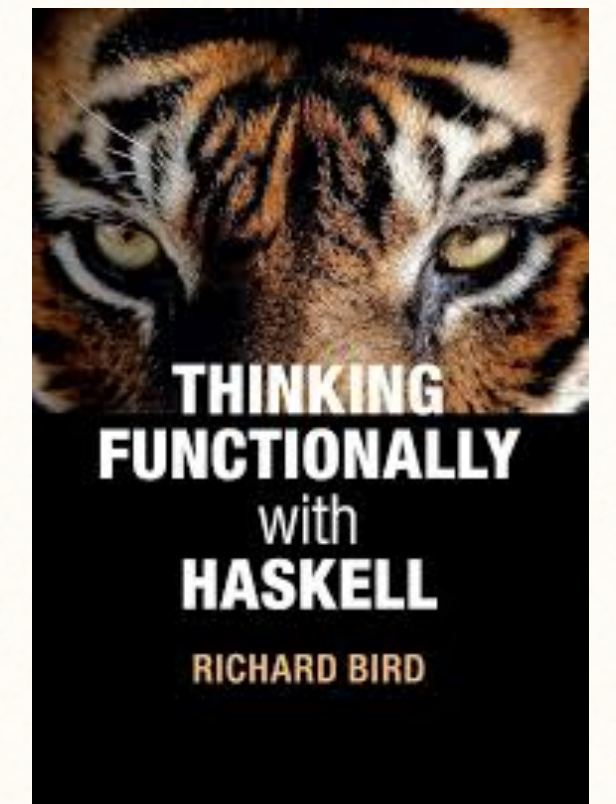
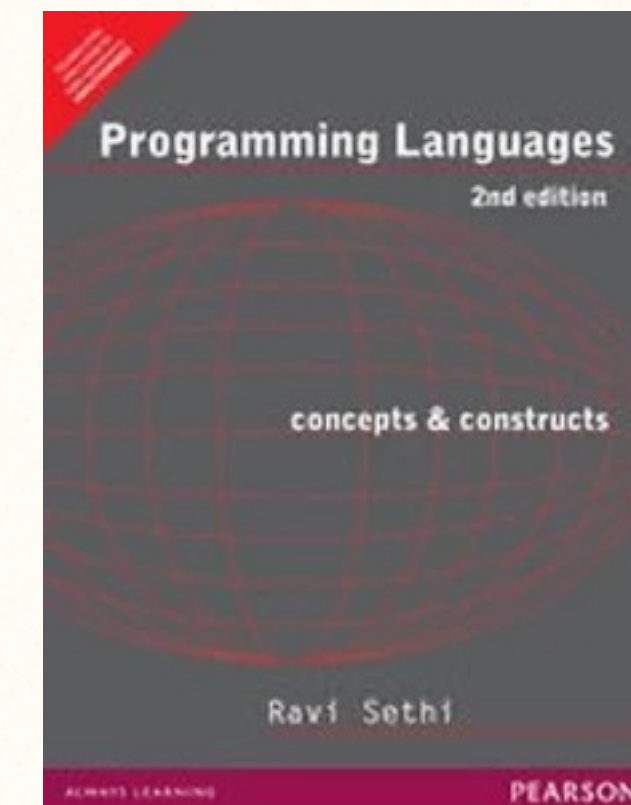
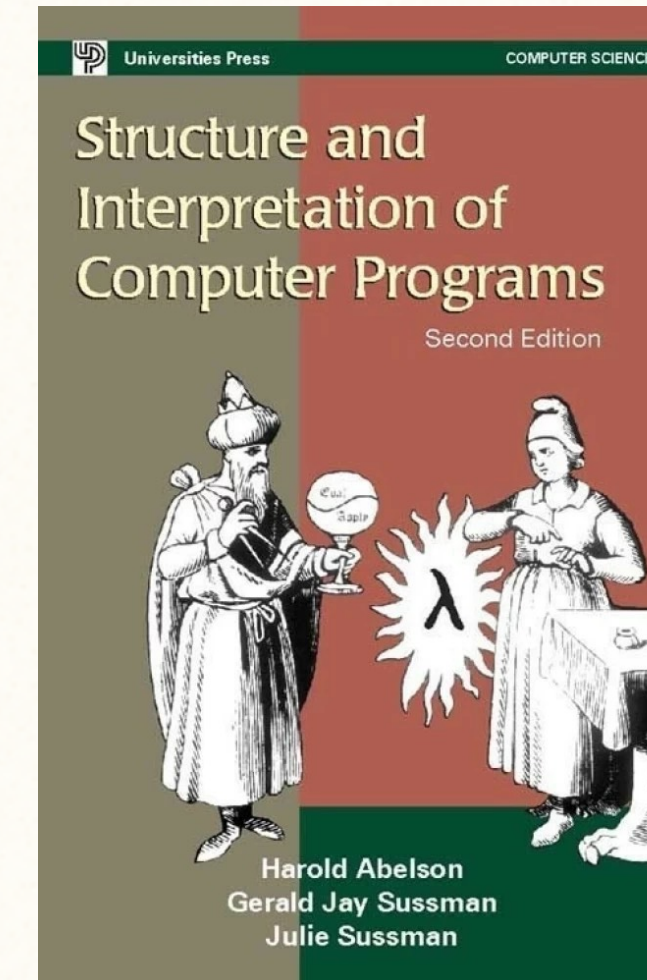
Learning Resources

Most
concepts and
Scheme

Lambda
calculus

Haskell,
types, proofs

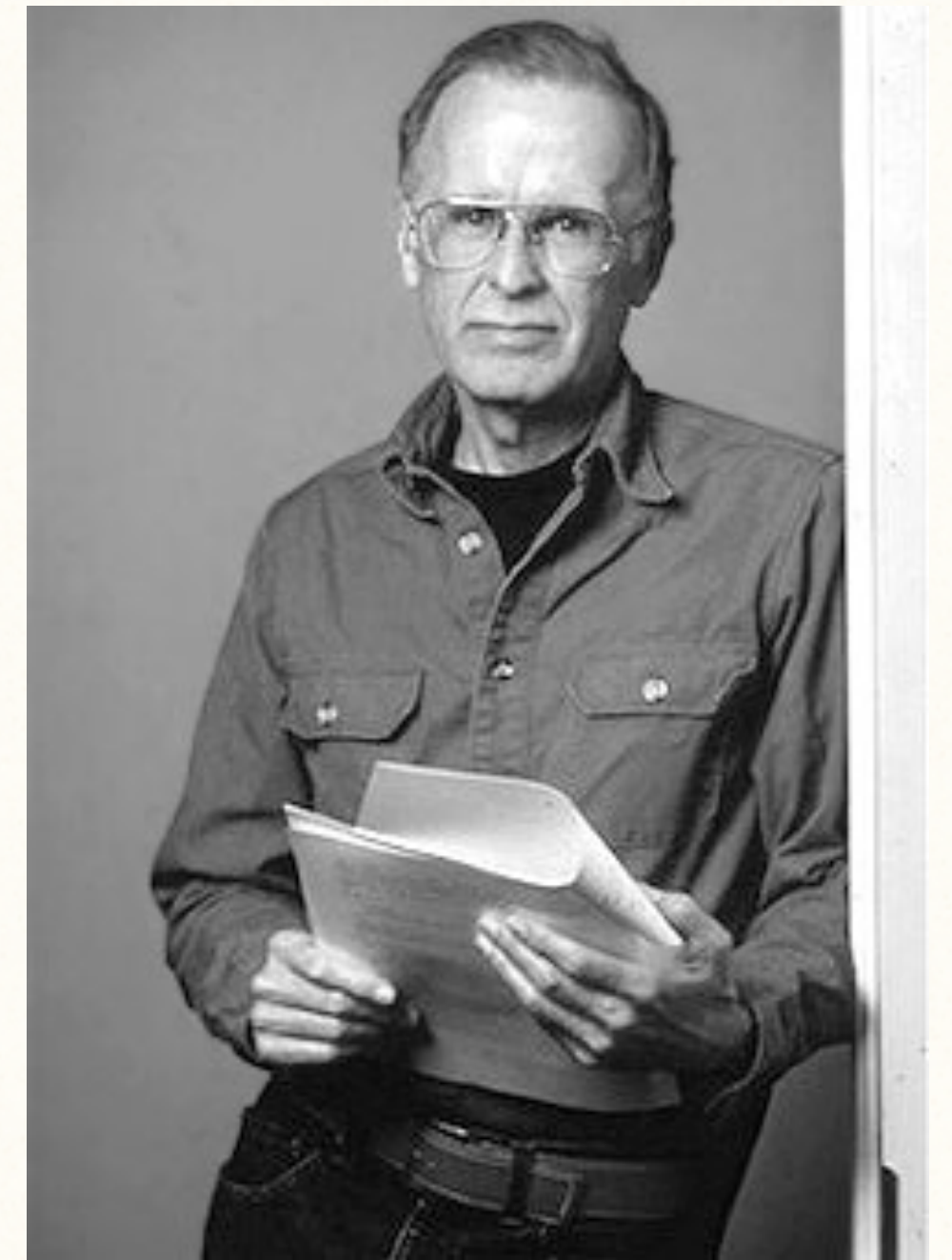
- Textbooks (**SICP**, PLCC, TFWH)
- Some videos and papers
- Some online resources
- Take notes in class (slides won't always be enough/available)
- Practice programs in the lab and at home



First Homework!

➤ Activity 1:

- Read first 7 pages of John Backus's Turing Award Lecture (don't worry if you don't understand everything)
 - Try finding out why was that lecture very interesting
 - Answer a **pop quiz** in the next class
-
- Install **DrRacket** with **sicp** package on your computer
 - <https://download.racket-lang.org/>



The *spirit* of this course

Dedication text of ***Structure and Interpretation of Computer Programs (SICP)***

Authors: **Harold Abelson and Gerald Jay Sussman with Julie Sussman (MIT)**

This book [course] is dedicated, in respect and admiration, to the spirit that lives in the computer:

“I think that it’s extraordinarily important that we in computer science keep fun in computing. When it started out, it was an awful lot of fun. Of course, the paying customers got shafted every now and then, and after a while we began to take their complaints seriously. We began to feel as if we really were responsible for the successful, error-free perfect use of these machines. I don’t think we are. I think we’re responsible for stretching them, setting them off in new directions, and keeping fun in the house. I hope the field of computer science never loses its sense of fun. Above all, I hope we don’t become missionaries. Don’t feel as if you’re Bible salesmen. The world has too many of those already. What you know about computing other people will learn. Don’t feel as if the key to successful computing is only in your hands. What’s in your hands, I think and hope, is intelligence: the ability to see the machine as more than when you were first led up to it, that you can make it more.”

—Alan J. Perlis (April 1, 1922 – February 7, 1990), first Turing Award winner



Unevaluated...



The thing you think a variable is — is wrong.